

• 오라클 5일차

• 복습 문제

- /* 문제 27 2007년도에 입사한 직원중에 job_id 'sa_rep'직원들의 인원 수,
- 총액급여, 평균급여, 최대급여, 최소급여를 출력해주세요. */
- select
- count(employee_id) as "인원 수",
- sum(salary) as "총액급여",
- avg(salary) as "평균급여",
- max(salary) as "최대급여",
- min(salary) as "최소급여"
- from hr.employees
- WHERE hire_date >= TO_DATE('2007-01-01', 'yyyy-mm-dd')
- AND hire_date < TO_DATE('2008-01-01', 'yyyy-mm-dd')
- AND job_id = 'SA_REP';

• Group by - 테이블의 행을 작은 그룹으로 나눌수 있는 절

- 9i R1 버전까지는 group by 가 sort group by로 수행하기 때문에 정렬된 결과 출력
- 9i R2 버전부터는 group by가 hash group by로 수행하기 때문에 정렬된 결과로 출력X
- select department_id, sum(salary)
- from hr.employees
- group by department_id;
- select job_id, count(*), sum(salary)
- from hr.employees
- group by job_id;
- 주의 점!
 - select절에 그룹함수에 포함되지 않은 개별 컬럼은 하나도 빠짐 없이 group by 절에 명시해야만 한다.
 - group by 절에는 열별칭, 위치표기법을 사용할 수 없다.
 - where 절에서는 그룹함수의 결과를 제한할 수 없다.
 - having 절을 이용하여 그룹함수의 결과를 제한한다.
 - select department_id, sum(salary)
 - from hr.employees
 - where last_name like '%i%'
 - group by department_id
 - having sum(salary) >= 10000
 - order by 1;
 - 절들의 순서는 꼭 지킬 것
 - 그룹함수를 두 번 중첩할 때 개별 컬럼이 있으면 오류

- /* 문제 28 2008년도 입사한 사원들의 job_id별 인원수 구하고 인원수가 많은 순으로 출력
- 문제 29 요일별 입사한 인원수를 출력
- 문제 30 월별 입사한 인원수 출력 */
-
- SELECT job_id, COUNT(*) AS 인원수
- FROM hr.employees
- WHERE TO_CHAR(hire_date, 'YYYY') = '2008'
- GROUP BY job_id
- ORDER BY COUNT(*) DESC;
-
- SELECT TO_CHAR(hire_date, 'DAY') AS 요일,
- COUNT(*) AS 인원수
- FROM hr.employees
- GROUP BY TO_CHAR(hire_date, 'DAY');
-
- SELECT TO_CHAR(hire_date, 'MM') AS 월,
- COUNT(*) AS 인원수
- FROM hr.employees
- GROUP BY TO_CHAR(hire_date, 'MM')
- ORDER BY 월;
-
- /* 문제 31 년도별 입사 인원수를 아래 화면과 같이 출력해주세요.
- Total 2001년 2002년 2003년
- 107 1 7 6 */
- SELECT
- COUNT(*) AS TOTAL,
- SUM(CASE WHEN TO_CHAR(hire_date, 'YYYY') = '2001' THEN 1 ELSE 0 END)
- AS "2001년",
- SUM(CASE WHEN TO_CHAR(hire_date, 'YYYY') = '2002' THEN 1 ELSE 0 END)
- AS "2002년",
- SUM(CASE WHEN TO_CHAR(hire_date, 'YYYY') = '2003' THEN 1 ELSE 0 END)
- AS "2003년"
- FROM hr.employees;
-
- select
- count(*) TOTAL,
- count (decode(to_char(hire_date,'yyyy'),'2001','x')) "2001년",
- count (decode(to_char(hire_date,'yyyy'),'2002','x')) "2002년",
- count (decode(to_char(hire_date,'yyyy'),'2003','x')) "2003년"
- from hr.employees;
-
- Join - 두개 이상의 테이블에서 내가 원하는 데이터를 가져오는 방법
 - 1. cartesian product
 - Join 조건이 생략, 잘못 만든 Join 조건
 - 첫 번째 테이블 행의 수와 두 번째 테이블의 행의 수가 곱해진다.
 - 2. equi join, inner join, simple join, 등가조인
 - join key 값이 일치하는 데이터만 추출하는 조인

- 컬럼의 정의가 명확하지 않으면 에러발생
 - 컬럼이름 앞에 테이블명을 접두어로 붙이면 컬럼의 모호성 해결
 - 테이블명을 컬럼이름 앞에 접두어를 사용하는 것보다 테이블의 별칭을 사용하게 되면 메모리 사용량을 줄일 수 있고 가독성도 좋아진다.
- -- M쪽 집합 1쪽 집합 Join 시 M쪽 집합의 랭 만큼 나오면 된다.(그 이상 나오면 안된다)
- select department_id from hr.employees;
- select department_id from hr.departments;
- select e.employee_id, e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id -- 조인조건
- and e.employee_id = 100; -- 비조인조건 술어
- select e.employee_id, e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id
- and e.last_name like '%k';
- select e.employee_id, e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id
- and e.job_id = 'IT_PROG';
- SELECT e.last_name, l.city
- FROM hr.employees e, hr.locations l, hr.departments d
- WHERE e.department_id = d.department_id -- 조인조건
- AND d.location_id = l.location_id; -- 조인조건
- /* 문제 32 사원들의 employee_id, country_name 출력하세요. */
- select e.employee_id, c.country_name
- from hr.countries c, hr.employees e, hr.departments d, hr.locations l
- where e.department_id = d.department_id
- AND d.location_id = l.location_id
- and l.country_id = c.country_id;
- outer join - 키 값이 일치되는 데이터, 또는 키값이 일치되지 않은 데이터도 출력하는
조인
 - 한쪽에만 (+) 표현
- select e.employee_id, e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id(+);
- select e.employee_id, e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id(+) = d.department_id;

- 양쪽에 + 할 경우 에러
- /* 문제 33 부서장 정보를 출력하세요.
- 문제 34 commission_pct에 null 이 아닌 사원들의 last_name,
- commission_pct, department_name을 출력해주세요.
- 문제 35 last_name에 소문자 'a'가 포함된 사원들의 last_name,
- department_name을 출력하세요 */
- select d.*, e.*
- from hr.employees e, hr.departments d
- where e.employee_id = d.manager_id;

- select e.last_name, e.commission_pct, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id(+)
- and e.commission_pct is not null;

- select e.last_name, d.department_name
- from hr.employees e, hr.departments d
- where e.department_id = d.department_id(+)
- and e.last_name like '%a%';

- 검증을 통해서 찾고한 값이 다 나왔는 지 확인을 할 필요가 있다.
- select count(*)
- from hr.employees
- where last_name like '%a%'

- non equi join 비등가조인
 - - equi join(=) 할 수 없는 다른 비교연산자(>, >=, <, <=)를 사용하는 조인의 유형
 - 값을 범위로 조인하려는 경우 많이 사용한다.
- -- worker
- SELECT employee_id, last_name, manager_id
- FROM hr.employees;

- -- manager
- SELECT employee_id, last_name
- FROM hr.employees;

- SELECT w.employee_id, w.last_name, m.employee_id, m.last_name
- FROM hr.employees w, hr.employees m
- where w.manager_id = m.employee_id(+);

- DROP TABLE job_grades purge;
- CREATE TABLE job_grades(
- grade_level varchar2(3),
- lowest_sal number,
- highest_sal number);

- INSERT INTO job_grades VALUES ('A',1000,2999);
- INSERT INTO job_grades VALUES ('B',3000,5999);

- INSERT INTO job_grades VALUES ('C',6000,9999);
- INSERT INTO job_grades VALUES ('D',10000,14999);
- INSERT INTO job_grades VALUES ('E',15000,24999);
- INSERT INTO job_grades VALUES ('F',25000,40000);
- commit;

- SELECT * FROM hr.job_grades;

- SELECT e.employee_id, e.salary, j.grade_level
- FROM hr.employees e, hr.job_grades j
- WHERE e.salary >= j.lowest_sal
- AND e.salary <= j.highest_sal;

- SELECT e.employee_id, e.salary, j.grade_level
- FROM hr.employees e, hr.job_grades j
- WHERE e.salary BETWEEN j.lowest_sal AND j.highest_sal;